

Bachelor of Information System

**IS2204 - Information System Security**

**Lecture – 2 : History and Hash**

Kasun De Zoysa  
kasun@ucsc.cmb.ac.lk



UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING



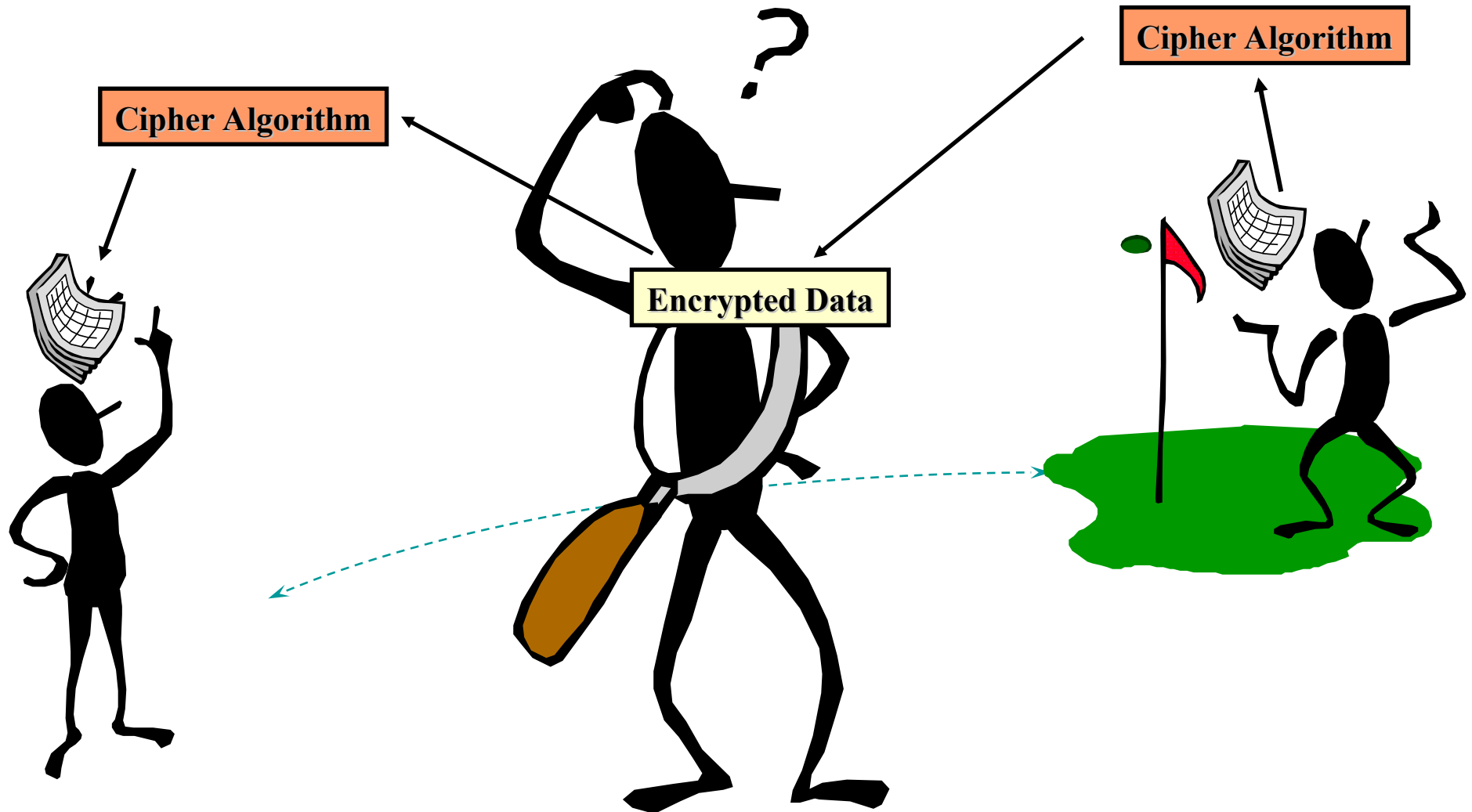
# Objectives - Cryptography

**The Cryptography domain addresses the principles, means, and methods of disguising information to ensure its integrity, confidentiality, authenticity and non-repudiation(?).**

# What You Should Know

- Basic concepts and terms within cryptography
  - Public and private key algorithms in terms of their applications and uses
  - Cryptography algorithm construction, key distribution, key management, and methods of attack
  - Applications, construction, and use of digital signatures
  - Principles of authenticity of electronic transactions and non-repudiation

# Basic Concept



# Definitions

- **Cryptography**
  - Art or science of secret writing
  - Protects sensitive information from disclosure
  - Storing and transmitting information in a form that allows it to be revealed only to those intended
  - Cryptosystem accomplishes this
  - Identifies the corruption or unauthorized change of information
  - Designed to make compromise too expensive or too time-consuming
- **Cryptanalysis**
  - art/science relating to converting ciphertext to plaintext without the (secret) key
  - descrambling without secret key ; art of breaking ciphers
  - Practice of defeating such attempts to hide info
- **Cryptology**
  - Includes both cryptography and cryptanalysis

# Definitions , continued

- **Encipher**
  - act of scrambling
- **Decipher**
  - descrambling with secret key
- **Key**
  - secret sequence governing en/deciphering

# Cryptography Basic

- **Why Encrypt?**
  - Protect stored information
  - Protect information in transmission
- Cryptography originally used for secrecy
- **Encryption** - process by which **plaintext** is converted to **ciphertext** using a **key**
- **Decryption** - process by which ciphertext is converted to plaintext (with the appropriate key)
- **plaintext** (cleartext)- intelligible data

# Cryptography Business Use

:

- Prevent unauthorized disclosure of information
- Prevent unauthorized access to information, computers, web sites, applications, etc.
- Detect tampering
- Detect injection of false data
- Detect deletion of data
- Prevent repudiation



# The goal of a cryptosystem

- **The goal of a cryptosystem is to provide**
- **Confidentiality** To ensure that unauthorized parties cannot access the data, message or information
- **Authenticity** To ensure that the source / sender of the data, message or information is identifiable
- **Integrity** To ensure that the data. Message or Information was not modified during transmission
- **Nonrepudiation** To ensure that either party cannot deny sending or receiving the data, message or information

# Cryptography History

- **Historic examples...**

- **Earliest cryptography**: an Egyptian scribe using non-standard hieroglyphics
- **Julius Caesar** (“Caesar Cipher”) Each plaintext letter is replaced by a letter some fixed number of positions further down the alphabet (e.g. Belgica (3 positions) → ehojld)
- The **Kama Sutra** recommends cryptography as 44<sup>th</sup> and 45<sup>th</sup> art (of 64) men and women should know

# Cryptography History

- **ENIGMA** Used by the Germans in WW2 – and the subsequent code-breaking activities at Bletchley park (still a popular subject of books and movies)
- 1976: **Public Key Cryptography** concept (Whitfield Diffie & Martin Hellman)
- 1977: first (*published*) practical PKC cryptosystem invented (RSA - Rivest, Shamir, Adleman)
- October 2000 **Rijndael is chosen as AES** (Advanced Encryption Standard)

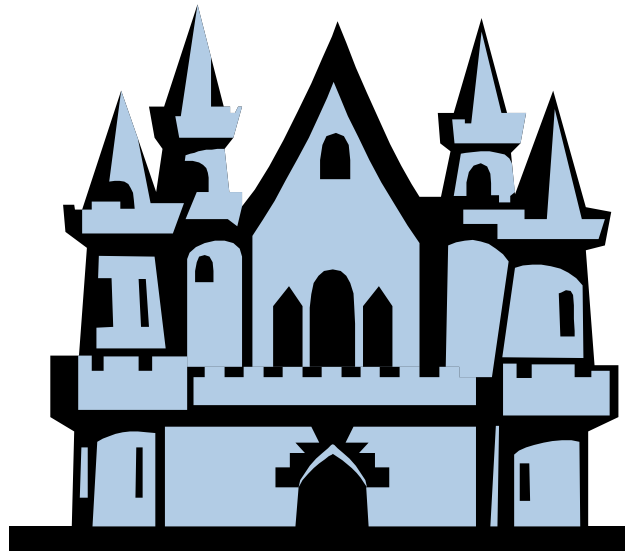


# The Caesar Cipher

Plain Text : **A B C D E F G H I J K L M N O P Q R S T U V W X Y Z**

Cipher Text : **D E F G H I J K L M N O P Q R S T U V W X Y Z A B C**

$$C_i = E(P_i) = P_i + 3$$



# Kamasutra

One of the earliest descriptions of encryption by substitution appears in the Kama-sutra, a text written in the 4th century AD by the Brahmin scholar Vatsyayana, but based on manuscripts dating back to the 4th century BC.

## How it work

The kamasutra generate list of 26 alphabet with no duplicate. Then divide by 2 row. Find for each letter of message text in table and choose the opposite of the letter

# Kamasutra

**for example:**

Key = G H A J R I O B E S Q C L F V Z T Y K M X W N U D P

**divide by 2 rows**

|   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| G | H | A | J | R | I | O | B | E | S | Q | C | L |
| F | V | Z | T | Y | K | M | X | W | N | U | D | P |

Given String = KAMASUTRA

K is at 2nd row and 5th column. Get the opposite of K that is I. Do each letter until the end

Cipher : IZOZNQJYZ

# Monoalphabetic Substitutions

Plain Text : **A B C D E F G H I J K L M N O P Q R S T U V W X Y Z**

Cipher Text : **K E Y G H I J K L M N O P Q R S T U V W X Y Z A B C**

## Letter Frequency



# One Time Pad / Vernam Cipher

- Invented in 1917 by Gilbert Vernam and Joseph Mauborgne. Usually implemented as a stream cipher using the XOR function.
- Key is used once and discarded by both sender and receiver.

Length of the Key character stream is equal to the message length.

- Not practical for large amounts of data (MB / GB).
- Pad is theoretically unbreakable by exhaustive brute force.
- Implementation uses a Key that consists of a set of random
- non-repeating characters.
- Each Key letter and Plaintext are added modulo 26 to each other and then converted back into a letter.



# One Time Pad / Vernam Cipher

Plain Text : V E R N A M C I P H E R  
Numeric Equivalent : 21 4 17 13 0 12 2 8 15 7 4 17  
+Random Number : 76 48 16 82 44 3 58 11 60 5 48 88  
= Sum : 97 52 33 95 44 15 60 19 75 12 52 105  
=Mod 26 : 19 0 7 17 18 15 8 19 23 12 0 1  
Cipher text : t a h r s p l t x m a b

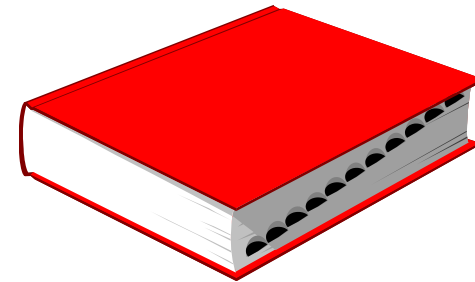
## Binary Vernam Cipher

Plain Text : 1 0 1 0 0 0 1 1 1 0 0 1 1 0 1  
⊕ Random Stream : 0 1 0 1 1 0 1 0 1 1 1 0 1 0 1  
Cipher text : 1 1 1 1 1 0 0 1 0 1 1 1 0 0 0

# Random Numbers

## 1. Truly Random numbers

- Books
- CD



## 2. Pseudo Random numbers

- Linear congruential random number generation

$$R_{i+1} = (a * R_i + b) \bmod n$$

# Encipherment Modes

- Stream Ciphers - Message broken into characters or bits and enciphered with a “key stream”
  - ✚ key stream - should be random and generated independently of the message stream
- Block ciphers process messages in blocks, each of which is then en/decrypted

# Block vs Stream Ciphers

- Block ciphers process messages in blocks, each of which is then en/decrypted
  - Like a substitution on blocks of characters
    - 64-bits or more
- Stream ciphers process messages a bit or byte at a time when en/decrypting
  - E.g. Vernam cipher, one time pad
- Many current ciphers are block ciphers

# Kerckhoff's Principle

The security of the encryption scheme must depend only on *the secrecy of the key and not on the secrecy of the algorithms.*

## Reasons:

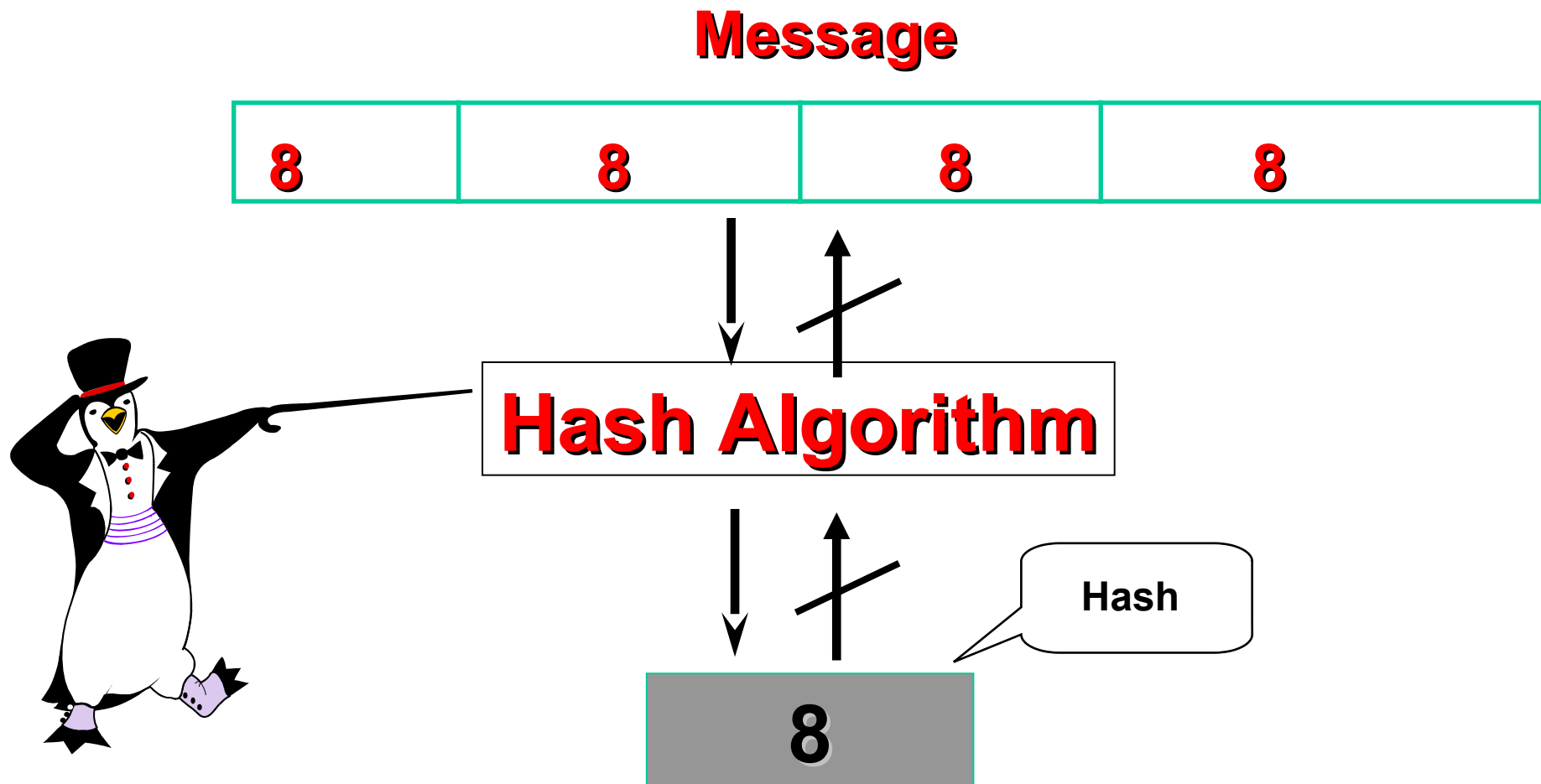
- Algorithms are difficult to change
- Cannot design an algorithm for every pair of users
- Expert review
- No security through obscurity!

# Hash Functions

- Condenses arbitrary message to fixed size
- Usually assume that the hash function is public and not keyed
  - MAC which is keyed (will discuss soon)
- Hash used to detect changes to message
- Can use in various ways with message
  - most often to create a password, digital signature etc.



# Hash Functions



# Simple Hash Functions

- There are several proposals for simple functions
- Some are based on XOR of message blocks
- Not secure since one can manipulate any message and either not change hash or manipulate the hash as well
- need a stronger cryptographic function



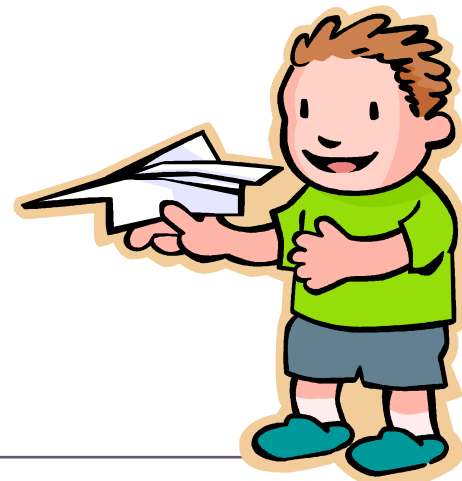


# Hash Function Properties

- A Hash Function produces a fingerprint of some file/message/data

$$h = H(M)$$

- condenses a variable-length message  $M$  to a fixed-sized fingerprint
- Assumed to be public



# Requirements for Hash Functions

- Can be applied to any sized message  $M$
- Produces fixed-length output  $h$
- Easy to compute  $h = H(M)$  for any message  $M$
- Given  $h$ , it is infeasible to find  $x$  s.t.  $H(x) = h$   
one-way property
- Given  $x$ , it is infeasible to find  $y$  s.t.  $H(y) = H(x)$   
weak collision resistance
- It is infeasible to find any  $x, y$  s.t.  $H(y) = H(x)$   
strong collision resistance

# MD5

- Designed by Ronald Rivest (the R in RSA)
- Latest in a series of MD2, MD4
- Produces a 128-bit hash value
- Until recently was the most widely used hash algorithm  
in recent times have both brute-force & cryptanalytic concerns
- Specified as Internet standard RFC1321

# Strength of MD5

- MD5 hash is dependent on all message bits
- Rivest claims security is good as can be
- Known attacks are:
  - Berson (92) attacked any 1 round using differential cryptanalysis (but can't extend)
  - Boer & Bosselaers (93) found a pseudo collision (again unable to extend)
  - Dobbertin (96) created collisions on MD compression function (but initial constants prevent exploit)
  - Crypto 2004 attacks on SHA-0 and MD5
- Conclusion is that MD5 has been shown to be vulnerable
- MD5 Collision Demo: <http://www.mscs.dal.ca/~selinger/md5collision/>

# Secure HASH Functions

- Purpose of the HASH function is to produce a "fingerprint."
- Properties of a HASH function  $H$  :
  1.  $H$  can be applied to a block of data at any size
  2.  $H$  produces a fixed length output
  3.  $H(x)$  is easy to compute for any given  $x$ .
  4. For any given block  $x$ , it is computationally infeasible to find  $x$  such that  $H(x) = h$
  5. For any given block  $x$ , it is computationally infeasible to find  $y$  with  $H(y) = H(x)$ .
  6. It is computationally infeasible to find any pair  $(x, y)$  such that  $H(x) = H(y)$

# Comparison - Modern HASH functions

| Algorithm Name | Comparison                                                                                                                                                                                                                                                                                                                       |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SHA-256        | <p>Widely used cryptographic hash function that generates a fixed-size output of 256 bits.</p> <p>It is considered secure and is commonly used in digital signatures, password storage, and other applications where data integrity and authenticity are critical.</p>                                                           |
| Argon2         | <p>Relatively new hashing algorithm that is designed to be resistant to both brute-force and side-channel attacks.</p> <p>It is commonly used for password storage and key derivation, and is considered to be one of the most secure hashing algorithms available.</p>                                                          |
| Yescript       | <p>Memory-hard hashing algorithm that is designed to be resistant to brute-force attacks and specialized hardware attacks, such as ASICs.</p> <p>It is commonly used for password storage and key derivation.</p> <p>Yescript is much slower than SHA-256, as it requires a large amount of memory to generate a hash value.</p> |

# Hashing Applications

- **Password storage:** Hashing algorithms are commonly used to store passwords securely.
- **Message authentication:** Hashing algorithms are often used to authenticate messages and ensure their integrity.
- **Digital signatures:** Hashing algorithms are used in digital signature schemes to verify the authenticity of a digital document or message.
- **Key derivation:** Hashing algorithms are often used to derive cryptographic keys from a password or other secret value.

# Hashing Applications

- **Data deduplication:** Hashing algorithms can be used to identify and remove duplicate data in storage systems.
- **Chain of custody (CoC):** Hashing algorithms can be used for tracking process that is used to establish the chronological history of the custody of digital evidence.
- **Blockchain technology:** Hashing algorithms are a critical component of blockchain technology, which is used for secure and decentralized record-keeping.
- **Many more ..**

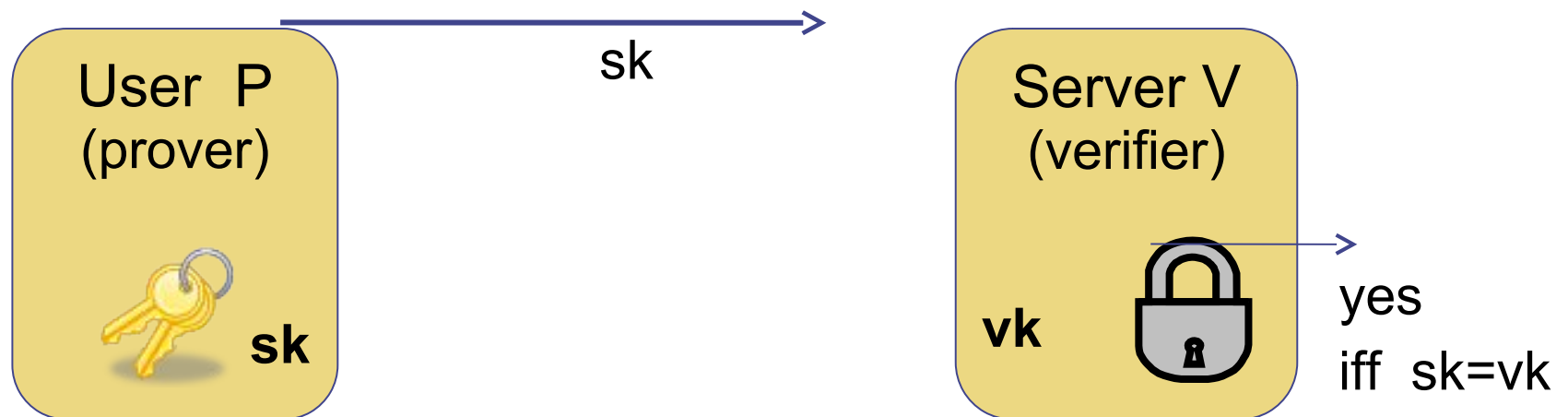


# Basic Password Protocol (incorrect version)

◆ **PWD**: finite set of passwords

◆ Algorithm G (KeyGen):

- choose rand pw in PWD. output  $sk = vk = pw$ .



# Basic Password Protocol (incorrect version)

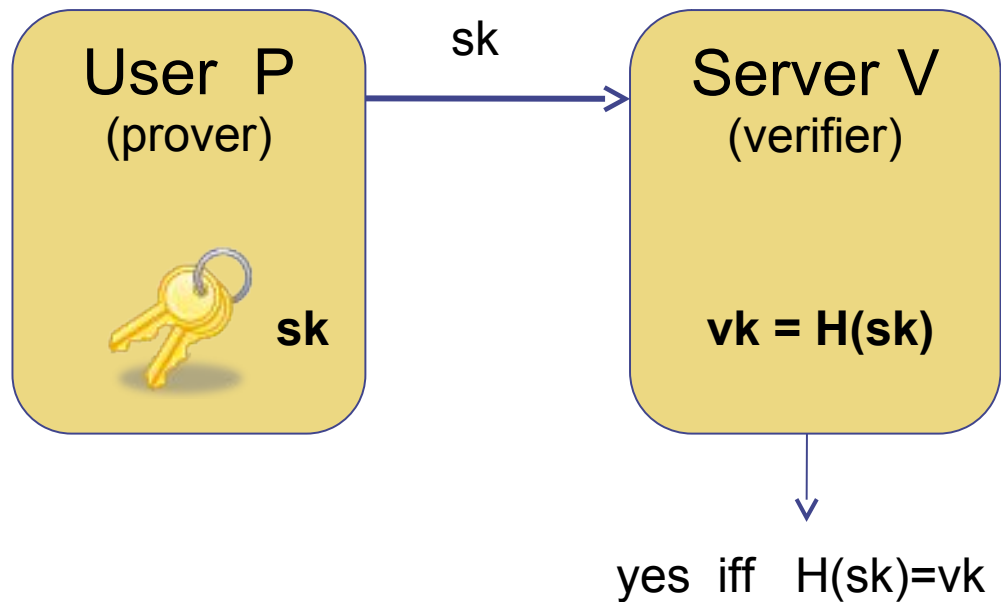
- ◆ Problem: VK must be kept secret
- Compromise of server exposes all passwords
  - Never store passwords in the clear!

password file on server

|       |                            |
|-------|----------------------------|
| Alice | $\text{pw}_{\text{alice}}$ |
| Bob   | $\text{pw}_{\text{bob}}$   |
| ...   | ...                        |

# Basic Password Protocol: version 1

H: one-way hash function from PWD to X  
“Given  $H(x)$  it is difficult to find  $y$  such that  $H(y)=H(x)$ ”



password file on server

|       |           |
|-------|-----------|
| Alice | $H(pw_A)$ |
| Bob   | $H(pw_B)$ |
| ...   | ...       |

# Weak Passwords and Dictionary Attacks

## ◆ People often choose passwords from a small set:

- The 6 most common passwords (sample of  $32 \times 10^6$  pwds):  
123456, 12345, Password, iloveyou, princess, abc123

('123456' appeared 0.90% of the time)

- 23% of users choose passwords in a dictionary of size 360,000,000

## ◆ **Online dictionary** attacks:

- Defeated by doubling response time after every failure
- Harder to block when attacker commands a bot-net

# Preventing Dictionary Attacks

## ◆ Public salt:

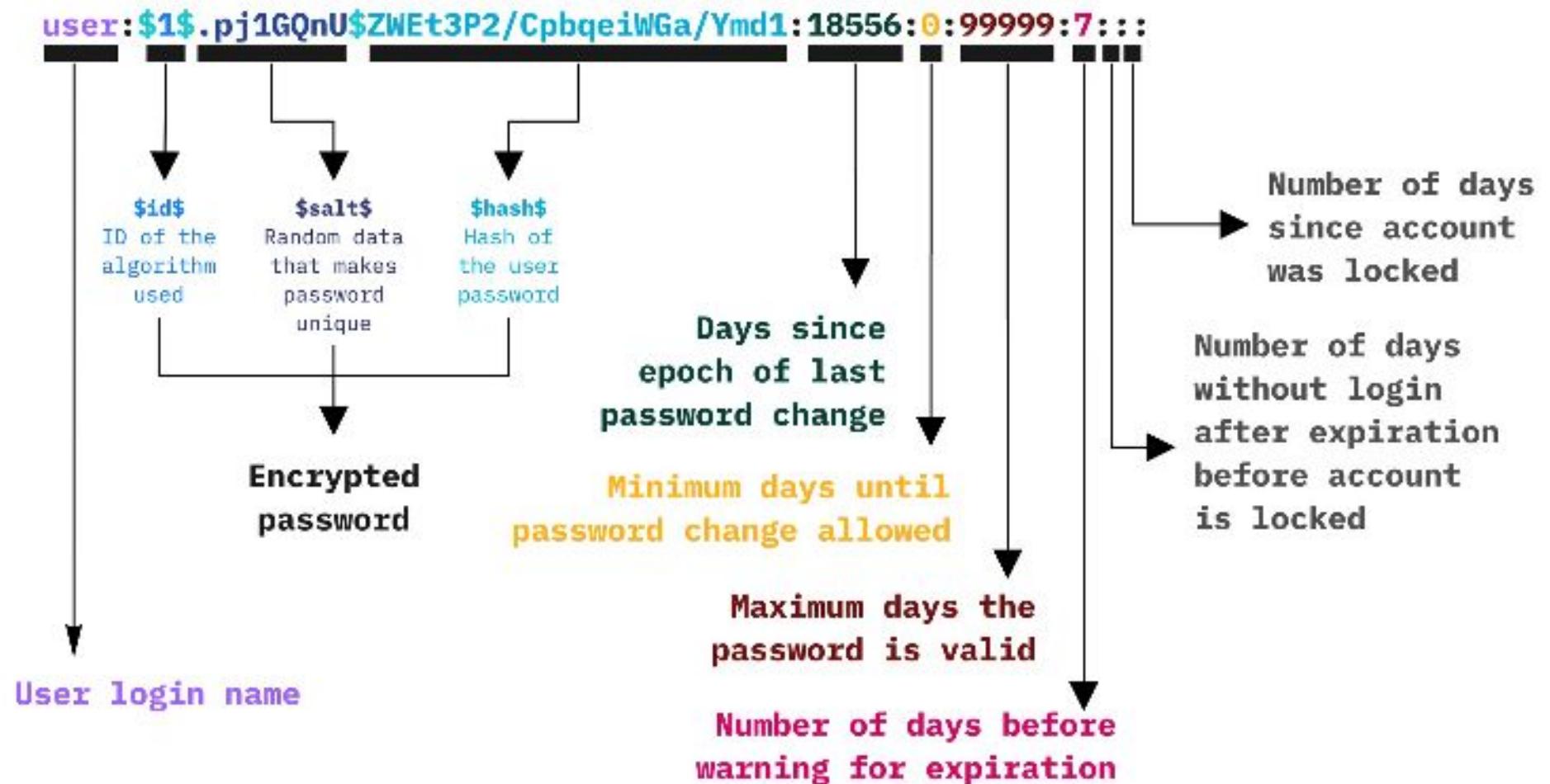
- When setting password, pick a random  $n$ -bit salt  $S$
- When verifying pw for  $A$ , test if  **$H(\text{pw}, S_A) = h_A$**

| id    | S     | h                     |
|-------|-------|-----------------------|
| Alice | $S_A$ | $H(\text{pw}_A, S_A)$ |
| Bob   | $S_B$ | $H(\text{pw}_B, S_B)$ |
| ...   | ...   | ...                   |

## ◆ Recommended salt length, $n = 64$ bits

- Pre-hashing dictionary does not help

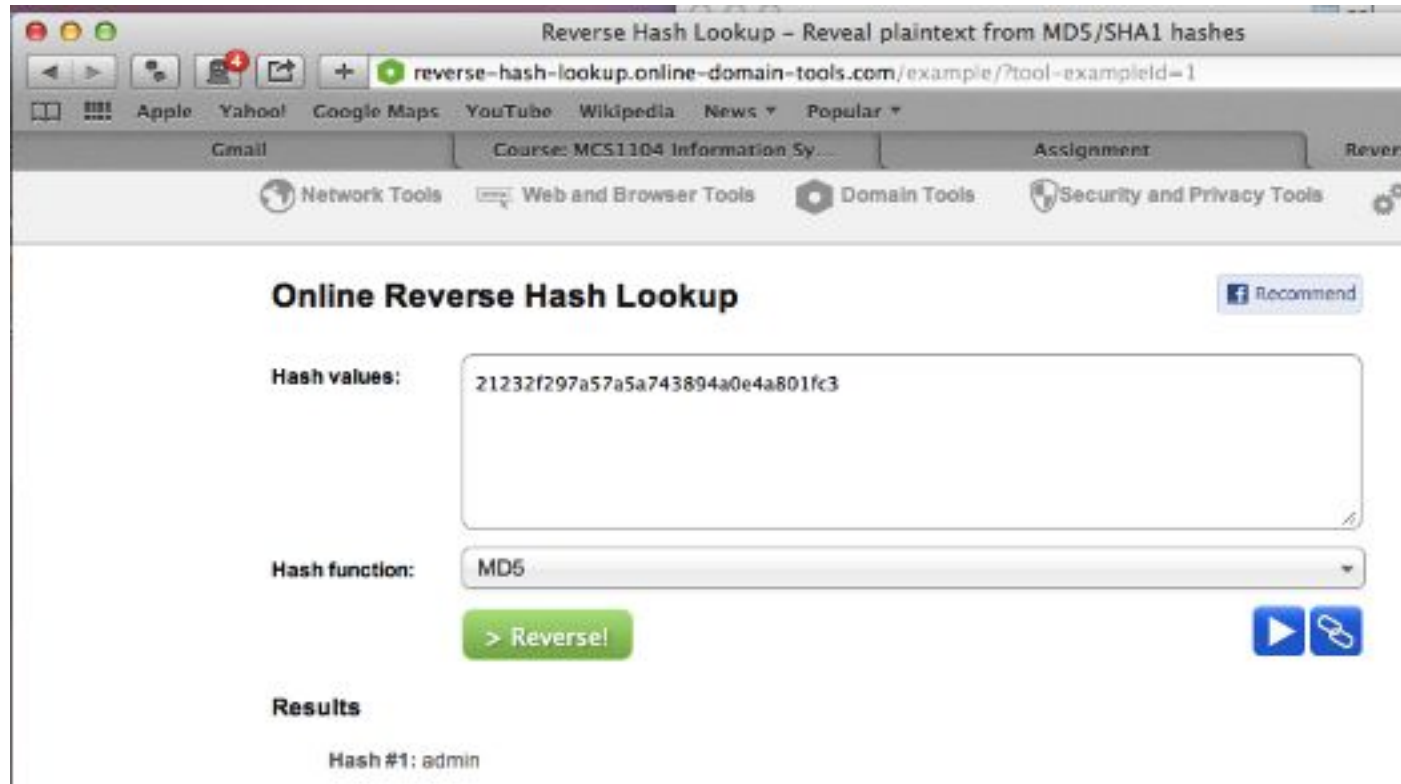
# /etc/shadow file



# Password Hashing Competition (PHC)

- In July 2015 PHC announced **Argon2** as a winner and gave special recognition to four of the finalists:
  - **Catena**, for its agile framework approach and side-channel resistance (Catena-v5.tar.gz)
  - **Lyra2**, for its elegant sponge-based design, and alternative approach to side-channel resistance (Lyra2-v3.tar.gz)
  - **Makwa**, for its unique delegation feature and its factoring-based security (Makwa-v1.tar.gz)
  - **yescrypt**, for its rich feature set and easy upgrade path from scrypt (yescrypt-v2.tar.gz)

# SHA1 Reverse Lookup



The screenshot shows a web browser window with the title "Reverse Hash Lookup - Reveal plaintext from MD5/SHA1 hashes". The address bar shows the URL "reverse-hash-lookup.online-domain-tools.com/example/?tool-exampleid=1". The browser's toolbar includes links to Apple, Yahoo!, Google Maps, YouTube, Wikipedia, News, and Popular. Below the toolbar, there are tabs for "Gmail", "Course: MCS1104 Information Sy...", "Assignment", and "Reverse". A navigation bar contains icons for "Network Tools", "Web and Browser Tools", "Domain Tools", and "Security and Privacy Tools". The main content area is titled "Online Reverse Hash Lookup" and features a "Recommend" button with a Facebook icon. A "Hash values:" label is followed by a text input field containing the hash "21232f297a57a5a743894a0e4a801fc3". Below this, a "Hash function:" label is followed by a dropdown menu set to "MD5". A green button labeled "> Reverse!" is positioned below the dropdown. To the right of the button are two blue icons: a play button and a link icon. Under the "Results" section, the text "Hash #1: admin" is displayed.

reverse-hash-lookup.online-domain-tools.com

```
echo -n 'kasun'| md5
```

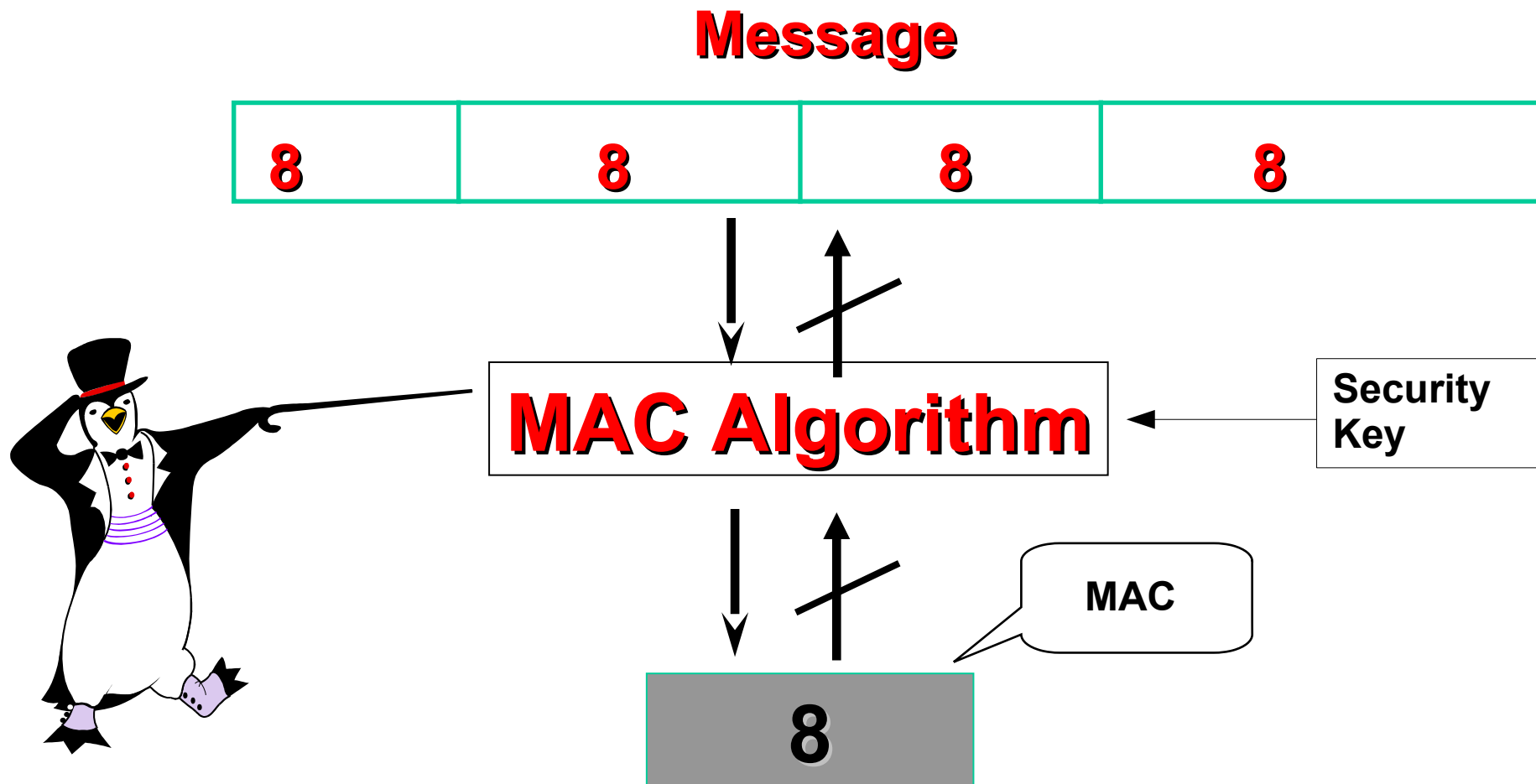


# Message Authentication Code (MAC)

- Generated by an algorithm that creates a small fixed-sized block depending on both message and some key
- need not be reversible
- Receiver performs same computation on message and checks if it matches the MAC
- Provides assurance that message is unaltered and comes from sender

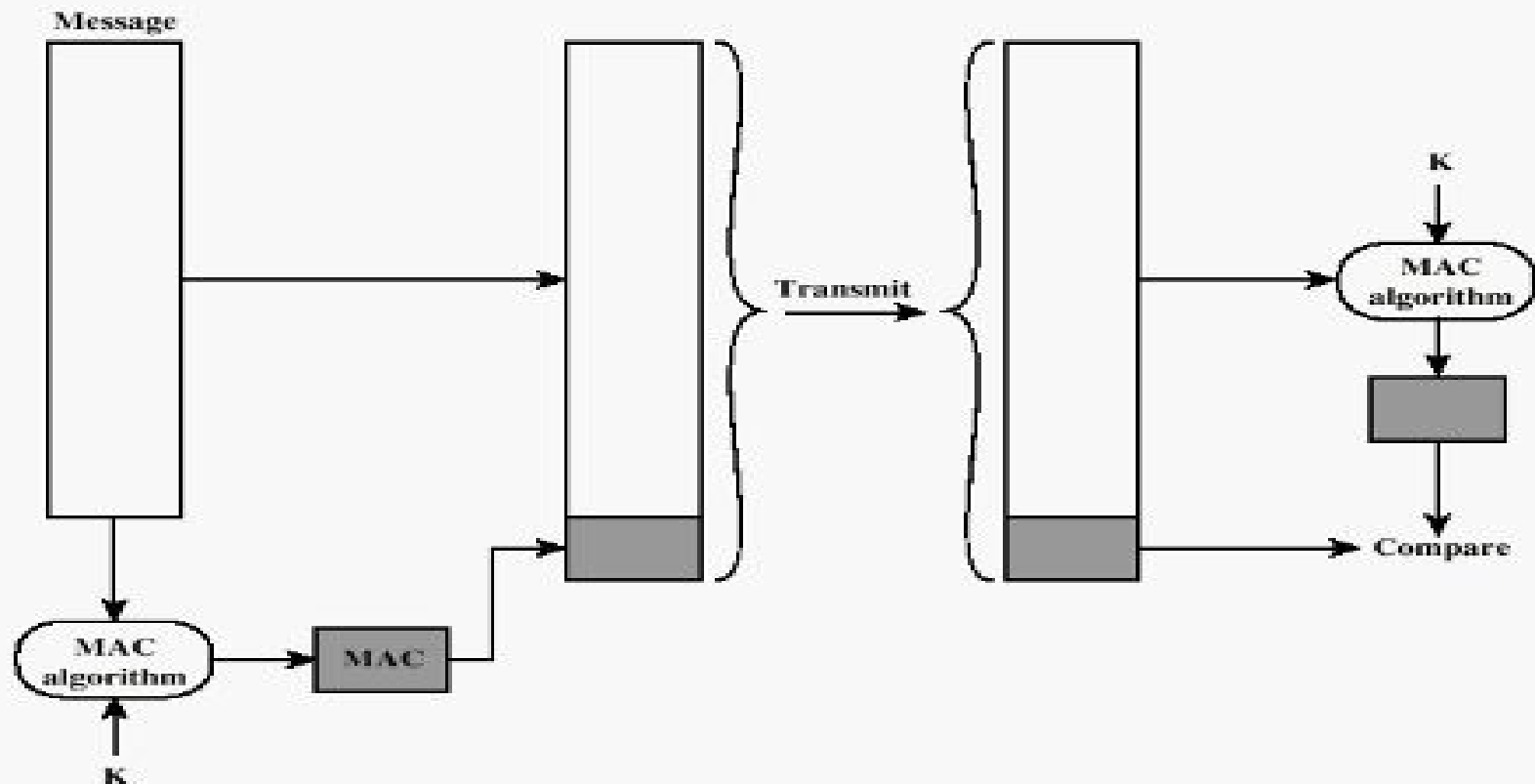


# Message Authentication Code (MAC)



# Approaches to Message Authentication

- Authentication Using Conventional Encryption
  - Only the sender and receiver should share a key
- Message Authentication without Message Encryption
  - An authentication tag is generated and appended to each message
- Message Authentication Code
  - Calculate the MAC as a function of the message and the key.  $MAC = F(K, M)$



**Message Authentication Using a Message Authentication Code (MAC)**

# Keyed Hash Functions (HMAC)

- Create a MAC using a hash function rather than a block cipher
  - because hash functions are generally faster
  - not limited by export controls unlike block ciphers
  - Hash includes a key along with the message
- Original proposal:
$$\textit{KeyedHash} = \textit{Hash}(\textit{Key}|\textit{Message})$$
  - some weaknesses were found with this
- Eventually led to development of HMAC

# Keyed Hash Functions (HMAC)

- Create a MAC using a hash function rather than a block cipher
  - because hash functions are generally faster
  - not limited by export controls unlike block ciphers
  - Hash includes a key along with the message
- Original proposal:  
$$\text{KeyedHash} = \text{Hash}(\text{Key}|\text{Message})$$
  - some weaknesses were found with this
- Eventually led to development of HMAC

# HMAC Design Criteria

- To use, without modifications, available hash functions.
- To allow for easy replaceability of the embedded hash function in case faster or more secure hash functions are found or required.
- To preserve the original performance of the hash function without incurring a significant degradation.
- To use and handle keys in a simple way.
- To have a well-understood cryptographic analysis of the strength of the authentication mechanism based on reasonable assumptions on the embedded hash function.

# HMAC Security

- know that the security of HMAC relates to that of the underlying hash algorithm
- attacking HMAC requires either:
  - brute force attack on key used
  - birthday attack (but since keyed would need to observe a very large number of messages)
- choose hash function used based on speed verses security constraints



# The birthday paradox

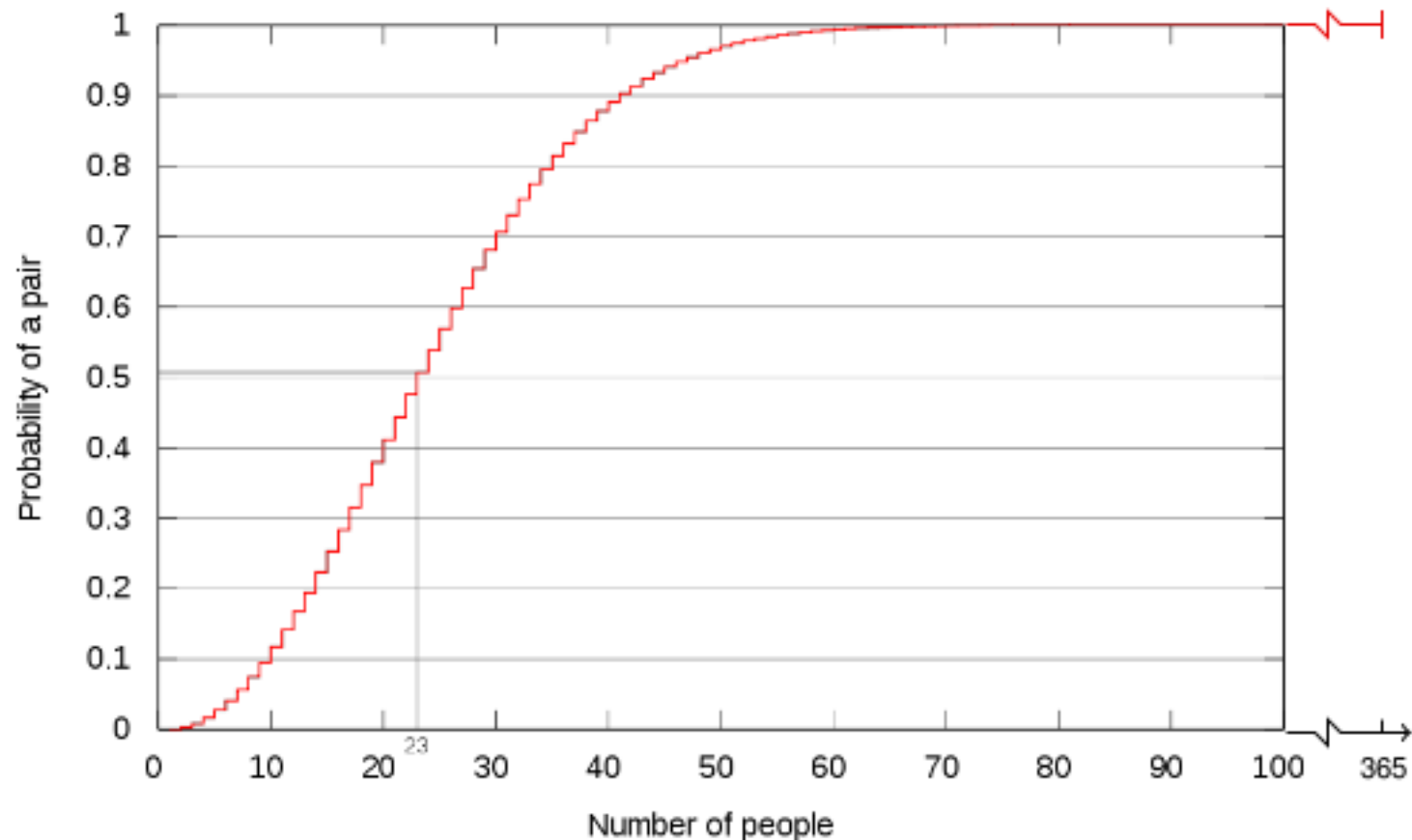
- How many people must there be in a room so that the probability of two of them having the same birthday is larger than 50%?
- One tends to be selfish in these cases and think: "The chance that another person has the same birthday as me is  $1/365$ . The chance that two other person has the same birthday as me is (almost)  $2/365$ .
- So, close to 183." But this is wrong! This calculation is correct when looking for matches to one specific person.

# The birthday paradox

- How many people must there be in a room so that the probability of two of them having the same birthday is larger than 50%?
- The correct calculation is the following: The chance that a second person does not have the same birthday as the first is  $364/365$ .
- If the two first do not have the same birthday, the chance that a third person does not have the same birthday as the two first is  $363/365$ .

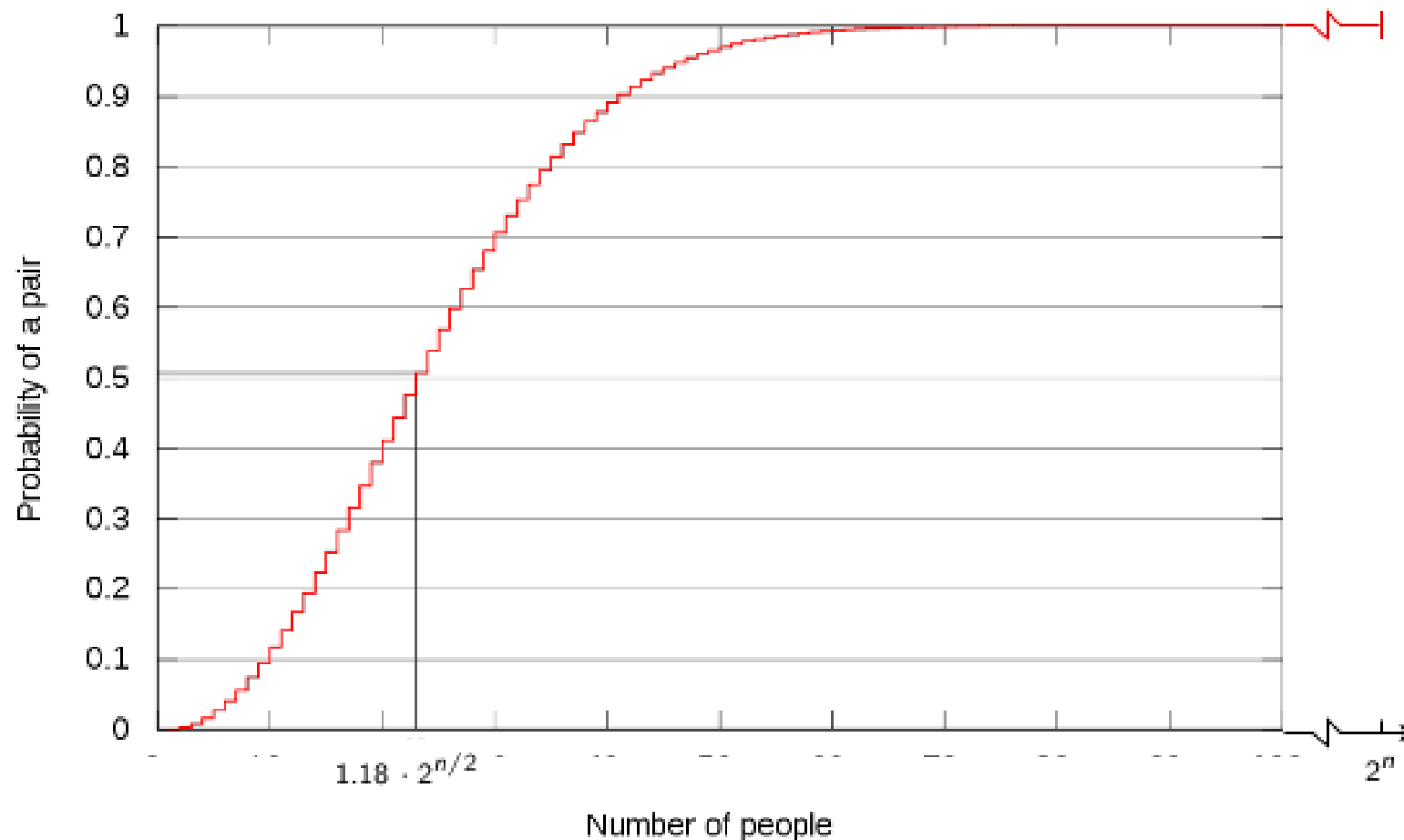
# The birthday paradox

How many people must there be in a room so that the probability of two of them having the same birthday is larger than 50%?

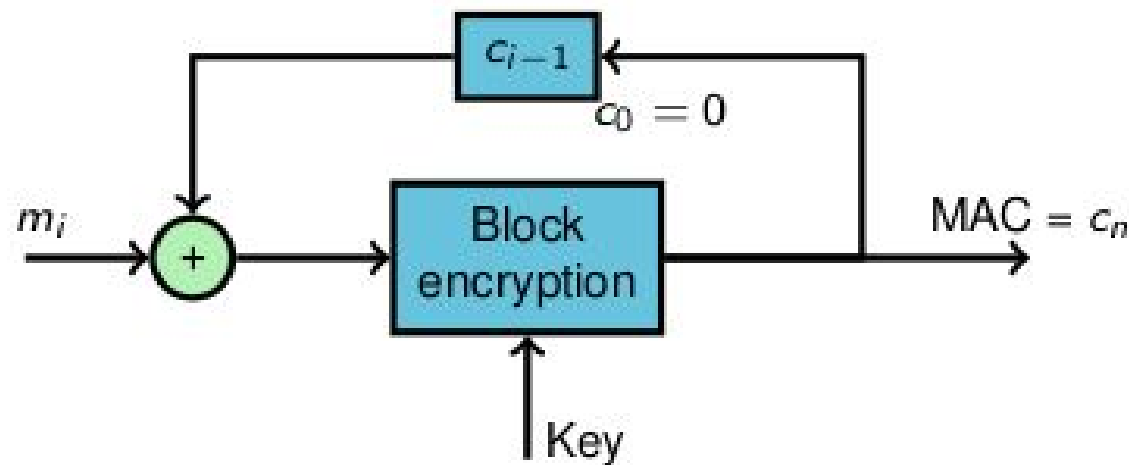


# The birthday paradox

How many people must there be in a room so that the probability of two of them having the same birthday is larger than 50%?



# Birthday attack on CBC-MAC



- Accumulate many message+MAC pairs, wait for collision
- This will take  $2^{64}$  steps for a 128-bit block cipher because of the birthday paradox (to find a collision, not to match a specific value)

# Discussion

